MSR '24

Student ID : 210042112

How do ML Projects use Continuous Integration Practices? An Empirical Study on GitHub Actions

João Helis Bernardo, Sérgio Queiroz de Medeiros, Uirá Kulesza(Federal University of Rio Grande do Norte), Daniel Alencar da Costa(University of Otago)
Presented by : Namisa Najah Raisa (Islamic University of Technology)

Problem Statement

CI practices are well-established in traditional software development but remain critically underexplored in Machine Learning (ML) projects. ML development introduces unique challenges : data preprocessing, model management, and intensive testing; that complicate CI adoption. Only **~37%** of ML projects use CI services, and whether they follow CI practices effectively is unknown.

Motivation

As ML adoption grows across industries, ensuring software quality through CI becomes increasingly critical. Without understanding how ML projects use CI, developers lack guidance for effective adoption. Uncovering these gaps can drive tailored CI approaches specifically designed for ML's unique demands.

Research Objectives

ML projects differ from traditional software due to data preprocessing, model diversity, and computational demands, complicating CI adoption (~37% of ML projects use CI services). This study addresses three research questions:

- **RQ1:** Differences in CI adoption (e.g., build duration, test coverage) between ML and non-ML projects?
- **RQ2:** Evolution trends of build duration and test coverage?
- **RQ3:** CI discussions in ML vs. non-ML developer communications?.

Dataset

- **185** open-source GitHub projects (**93 ML, 92 non-ML**)
- Using GitHub Actions CI workflows (**≥100 runs, ≥6 months** history).
- **Metrics:** commit activity, build duration (successful runs), time to fix broken builds, test coverage (Coveralls/Codecov).
- **Qualitative: 719** pull requests (**365 ML, 354 non-ML**).
- Project sizes by **LOC:** small (<10k), medium (10k-100k), large (>100k).

	projects		median age		contributors		commits		pull requests		coverage builds		CI builds	
size	ML	nonML	ML	nonML	ML	nonML	ML	nonML	ML	nonML	ML	nonML	ML	nonML
small	8	14	5.6	7.5	28	73	5497	6103	2425	4511	811	1054	3200	4049
medium	44	38	5.4	9	69	368	37081	34545	20900	31766	6736	7647	35678	32983
large	41	40	6.6	8.7	187	624	100700	128988	110704	84506	28770	30086	161577	111992
total	93	92	5.7	8.4	102	387	143278	169636	134029	120783	36317	38787	200455	149024

Table I: Projects general information.

Methodology

Project selection: Started from curated dataset; filtered for 2023 updates, GitHub Actions CI workflows (triggers: push/PR; terms: test/CI), ≥100 runs, ≥6 months history, balanced ML/non-ML.

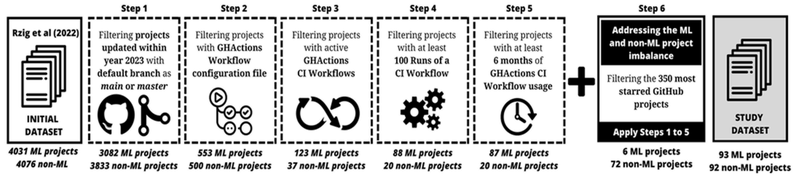


Fig. 1: An overview of the project selection process.

Data collection: GitHub API for repository and CI metadata (workflows, runs, commits, pull requests); CodeTabs for LOC extraction; MySQL for storage; analysis conducted in **30-day periods** following a **15-day stabilization phase**.

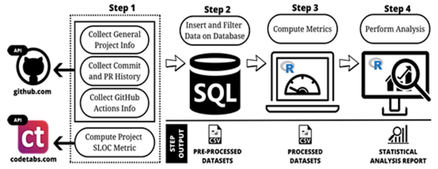


Fig. 2: An overview of the data collection process.

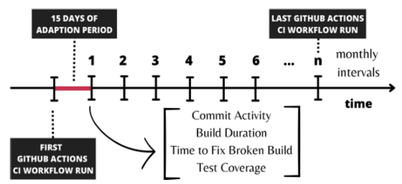


Fig. 3: Periods of Analysis.

- **RQ1:** Median metric values per project; Mann–Whitney U test ($p < 0.05$); Cliff's delta effect size.
- **RQ2:** Dynamic Time Warping clustering; gap statistic for optimal cluster count; trend categories (increasing/decreasing/maintaining) from cluster centroids.
- **RQ3:** Inductive thematic analysis of stratified PR samples (95% confidence level); open and axial coding by multiple researchers.

Results

RQ1: Small ML builds: **10.3** vs. **0.81** min (p=0.00298, Cliff's δ=large); Medium: **13** vs. **5.54** min (p=0.000566, δ=medium). Difference persists across language types; data overhead is the primary driver. Medium ML coverage: 83% vs. 94% (p=0.00676, δ=large)

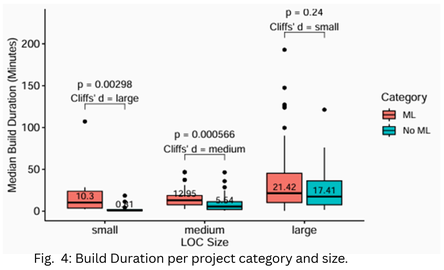


Fig. 4: Build Duration per project category and size.

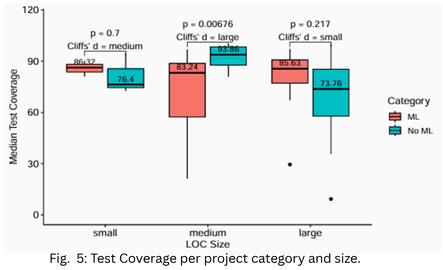


Fig. 5: Test Coverage per project category and size.

RQ2: Small/medium ML: higher increasing build trends (75%/61.4% vs. 35.7%/44.7% non-ML); large ML lower (51.2% vs. 65%). Both maintain test coverage; 46% medium ML <75%.

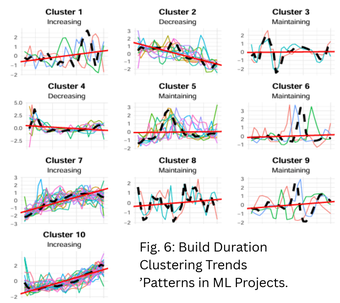


Fig. 6: Build Duration Clustering Trends 'Patterns in ML Projects.

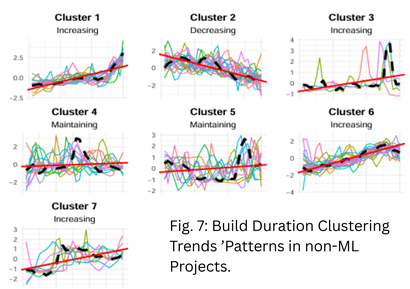
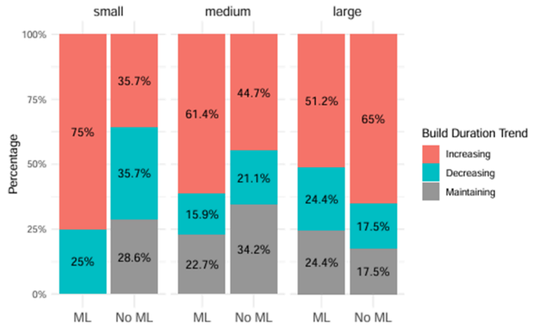


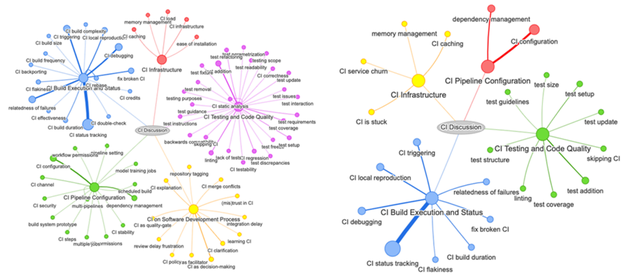
Fig. 7: Build Duration Clustering Trends 'Patterns in non-ML Projects.

Results Contd.



Fig, 8: Clustering trends per project category and size.

RQ3: Common themes: CI execution/status, infrastructure, configuration, testing. ML: more themes (73 vs. 23), unique (mistrust in CI, CI as quality gate); "**relatedness of failures**" (false positives) ranks 3rd in ML, 8th non-ML.



(a) CI discussions in ML projects. (b) CI discussions in non-ML projects.

Fig. 9: CI discussions in ML and non-ML projects.

Conclusion & Implications

Conclusion: ML projects exhibit longer builds (small: 10.3 vs. 0.81 min; medium: 13.0 vs. 5.54 min), lower medium-sized test coverage (83% vs. 94%), increasing build trends (small: 75% vs. 35.7%), and more diverse CI discussion themes (73 vs. 23), especially false positives (ranked 3rd vs. 8th).

Implications:

- Parallelize/cache workflows to curb build growth.
- Develop ML-tailored testing acknowledging data complexity.
- Audit CI false positives proactively.

Contributions

1. First Empirical Comparison of CI Practices Between ML and Non-ML Projects
2. Evidence That ML Projects Suffer From Longer Build Durations
3. Evidence of Lower Test Coverage in Medium-Sized ML Projects
4. Identification of False Positives as an ML-Specific CI Problem
5. A Foundation for Future SE4ML Research